Marc Gamell

Informe de la pràctica 2 Interrupcions i Activitat Complementària

Introducció

Aquesta segona pràctica de l'assignatura Processadors Digitals introdueix un dels mecanismes més importants i potents en la programació de microcontroladors: les **interrupcions**. Es deixa enrere el mètode de consulta contínua, conegut com a *polling* (on el processador perd temps i recursos comprovant periòdicament l'estat d'una entrada), per passar a un model basat en esdeveniments, molt més eficient i ràpid.

1.1. Objectius

L'objectiu principal de la pràctica és comprendre en profunditat el funcionament de les interrupcions. Per assolir-ho, l'exercici es divideix en diverses fases amb els següents objectius específics:

- **Implementació d'interrupcions per maquinari (GPIO):** Configurar un pin d'entrada perquè un esdeveniment físic extern (com prémer un botó) aturi temporalment el flux normal del programa i executi una acció immediata.
- **Gestió d'interrupcions programades (Timers):** Utilitzar els temporitzadors interns del xip ESP32-S3 per generar interrupcions exactes basades en el temps, permetent comptar segons amb precisió sense bloquejar el processador.
- **Filtratge de senyals (Debounce):** Aplicar solucions per programari per resoldre problemes físics, com els rebots mecànics dels polsadors que generen falses lectures múltiples.
- **Concurrència i disseny assistit per IA:** Combinar el control de LEDs i botons de forma simultània utilitzant codi generat per intel·ligència artificial (ChatGPT) per estudiar diferents arquitectures de programació.

1.2. Material utilitzat

Per dur a terme els diferents muntatges i proves d'aquesta pràctica, s'ha utilitzat el següent material:

Maquinari (Hardware):

- **Microcontrolador:** Placa de desenvolupament basada en el xip ESP32-S3.
- **Components electrònics:** Un LED i diversos polsadors (botons) per interactuar físicament amb els pins del microcontrolador.
- **Muntatge:** Placa de proves (protoboard) i cablejat de connexió per muntar els circuits externs.

Programari (Software):

- **Entorn de treball:** Visual Studio Code amb l'extensió PlatformIO, utilitzant el framework d'Arduino (C++).

- **Eines de suport:** Ús de ChatGPT com a assistent de programació per a l'activitat de disseny addicional.

1.3. Conceptes Claus: Què és una interrupció?

A nivell teòric, una interrupció és un senyal d'avís que suspèn l'activitat normal del microprocessador per atendre un esdeveniment electrònic urgent de forma gairebé instantània.

Quan es produeix aquest esdeveniment, el processador posa en pausa el codi principal de manera ordenada i salta a executar una funció especial anomenada **ISR** (Rutina de Servei d'Interrupció). En el cas de l'ESP32, es recomana allotjar aquestes rutines a la memòria RAM interna mitjançant l'identificador IRAM_ATTR, fet que garanteix una velocitat d'execució molt superior a la de la memòria Flash.

Un cop l'ISR ha finalitzat la seva tasca curta i precisa, el processador reprèn la seva activitat anterior exactament en el punt on l'havia deixada, com si no hagués passat res. En aquesta pràctica s'exploraran dos dels tres tipus d'esdeveniments que poden disparar aquestes ISR: els esdeveniments físics de maquinari (canvis d'estat a la pota d'un botó) i els esdeveniments programats per temporitzador.

Desenvolupament i Implementació (Pràctica A)

Aquesta pràctica s'ha centrat en comprendre i implementar el mecanisme d'interrupcions físiques (per maquinari) com a alternativa al mètode tradicional de *polling* (consulta contínua). Mitjançant una interrupció, el processador deixa de costat la seva tasca actual de forma immediata per atendre un esdeveniment extern (com la pulsació d'un botó), millorant dràsticament l'eficiència i la velocitat de resposta.

1. Configuració de l'Entorn (platformio.ini)

L'arxiu de configuració platformio.ini manté la mateixa estructura bàsica utilitzada a la pràctica anterior. Defineix l'entorn de treball ([env:esp32-s3-devkitm-1]), l'arquitectura (espressif32), la placa concreta (esp32-s3-devkitm-1) i l'ús del framework d'Arduino.

L'única línia destacable per al funcionament d'aquesta pràctica és:

- **monitor_speed = 115200**: Estableix la velocitat de comunicació sèrie a 115200 bauds. És imprescindible que aquest valor coincideixi amb la inicialització del codi principal per poder visualitzar correctament per pantalla els missatges relacionats amb la interrupció.

2. Codi Principal: Pràctica A - Interrupció per GPIO (main.cpp)

L'objectiu d'aquest codi és detectar les pulsacions d'un botó utilitzant una interrupció associada a un pin GPIO. A més, s'ha implementat una millora de filtratge (debounce per programari) per evitar falses lectures.

Estructures de dades i variables:

- **struct Button {...}**: Es defineix una estructura de dades per agrupar tota la informació relacionada amb el botó. Dins d'aquesta estructura s'hi troben:
    - PIN: El número de pin físic.
    - numberKeyPresses: Un comptador total de pulsacions.
    - pressed: Una variable booleana (true/false) que actua com a "bandera" (flag) per avisar al bucle principal que el botó s'ha premut.
    - lastMillis: Una variable per emmagatzemar l'instant de temps de l'última pulsació (utilitzada per al filtratge).
- **Button button1 = {20, 0, false};**: Es crea una instància d'aquesta estructura, assignant el botó al **pin 20** i inicialitzant els comptadors a zero i l'estat a fals.

Rutina de Servei d'Interrupció (ISR): Aquesta és la funció especial que s'executa automàticament i de forma fulgurant quan es detecta un canvi físic al pin.

- **void IRAM_ATTR isr() { ... }**: L'etiqueta IRAM_ATTR és una directiva molt important d'Espressif. Indica al compilador que aquesta funció s'ha de carregar a la memòria RAM interna del xip en lloc de la memòria Flash. La RAM és molt més ràpida de llegir, la qual cosa garanteix que la interrupció s'atengui i s'executi gairebé instantàniament.
- **uint32_t now = millis();**: Obté el temps actual en mil·lisegons des que la placa es va engegar.
- **if (now - button1.lastMillis > 200) { ... }**: Aquesta línia implementa un **filtre antirebots (debounce)**. Comprova si han passat més de 200 mil·lisegons des de l'última interrupció. Quan es prem un botó físic, els contactes metàl·lics reboten generant múltiples senyals elèctrics molt ràpids. Si la diferència de temps és més gran de 200ms, s'accepta com una pulsació real i humana; si és inferior, s'ignora (és un rebot).
- Dins de l'IF, s'incrementa el comptador (numberKeyPresses += 1;), s'activa la bandera avisant que hi ha hagut una pulsació (pressed = true;) i s'actualitza el temps de l'última pulsació.

Funció setup():

- **pinMode(button1.PIN, INPUT_PULLUP);**: Configura el pin del botó com a entrada i activa la resistència de *pull-up* interna del microcontrolador. Això assegura que el pin llegeixi un valor ALT (HIGH) estable quan el botó no està premut.
- **attachInterrupt(button1.PIN, isr, FALLING);**: Aquesta és la instrucció clau. Enllaça físicament l'interrupció al pin 20. Li diu al processador que es quedi vigilant aquest pin i que, quan detecti un flanc de baixada (FALLING - és a dir, quan el voltatge passa d'ALT a BAIX al prémer el botó), aturi el que estigui fent i executi la funció isr.

Funció loop():

El bucle principal aquí ja no s'encarrega d'esperar o mirar contínuament si el botó està premut (polling). La seva única feina relacionada amb el botó és revisar ràpidament l'estat de la bandera.

- **if (button1.pressed) { ... }**: Si la interrupció ha canviat el valor d'aquesta bandera a true, entra a la condició.

- **Serial.printf("Button 1 has been pressed %u times\n", button1.numberKeyPresses);**: Mostra **pel terminal de sortida** un missatge indicant quantes vegades s'ha premut el botó en total.
- **button1.pressed = false;**: Restableix la bandera a false per no tornar a imprimir el missatge fins que hi hagi una nova pulsació.

Desconnexió de la interrupció (Comprovador de Temps): Al final del loop() s'inclou un bloc de codi per demostrar com desactivar una interrupció en temps d'execució.

- **if (millis() - lastMillis > 60000) { ... }**: Aquest bloc actua com un temporitzador per programari. Comprova si ha passat un minut (60000 mil·lisegons) des de l'inici.
- **detachInterrupt(button1.PIN);**: Desconnecta formalment la interrupció del pin 20. A partir d'aquest moment, per molt que premem el botó, el processador ignorarà els canvis de voltatge en aquest pin i no executarà la funció isr.
- Finalment, envia un missatge confirmant aquesta desconnexió: **Serial.println("Interrupt Detached!");** **pel terminal de sortida**.

Anàlisi del Temps Lliure del Processador (Interrupció per GPIO)

L'anàlisi de l'ocupació de la CPU en aquest programa revela un canvi de paradigma fonamental respecte a la Pràctica 1, passant d'un model d'execució bloquejant a un model basat en esdeveniments (o *event-driven*).

El problema del codi anterior (Pràctica 1): En la primera pràctica, teníem un 99% de "temps lliure", però era un **temps mort o bloquejat** a causa de la instrucció delay(). El processador estava aturat comptant el temps i era incapaç de fer absolutament res més, com per exemple, llegir ràpidament l'estat d'un botó d'emergència. Aquesta necessitat de reaccionar de forma immediata i sense demores a esdeveniments externs és el que justifica l'ús d'interrupcions.

L'eficiència del nou codi (Pràctica 2 - GPIO):

En el programa actual (main.cpp), si analitzem la funció loop(), podem veure que **no hi ha cap instrucció de pausa**. El bucle es limita a fer dues comprovacions rapidíssimes:

1. Mirar si la bandera button1.pressed és certa.
2. Mirar si ha passat més d'un minut per desconnectar la interrupció.

Com que avaluar condicions és una de les coses que els microcontroladors fan més ràpid, el loop() s'executa contínuament milers de vegades per segon sense aturar-se mai.

Com es distribueix el temps ara?

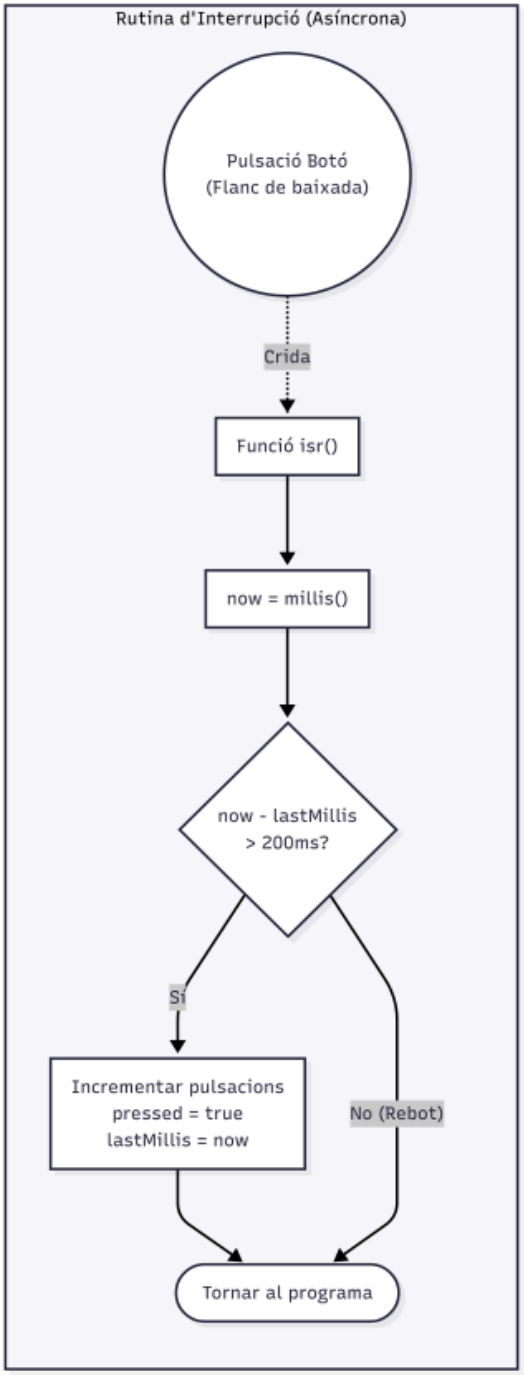
- **Temps de Polling (Comprovació manual): 0%.** El processador no perd gens de temps comprovant periòdicament (fent *polling*) si el botó s'ha premut o no.
- **Temps d'execució de la interrupció (ISR): < 0,1%.** Només quan algú prem físicament el botó, el maquinari avisa al processador. Aquest atura el bucle loop() un

instant, executa la funció isr() (que al ser curta i estar carregada a la memòria RAM s'executa gairebé a l'instant) i retorna immediatament al lloc on s'havia quedat.

- **Temps lliure (Actiu i Disponible): > 99,9%.**

Diagrames de flux i temps (A)

Diagrama de flux



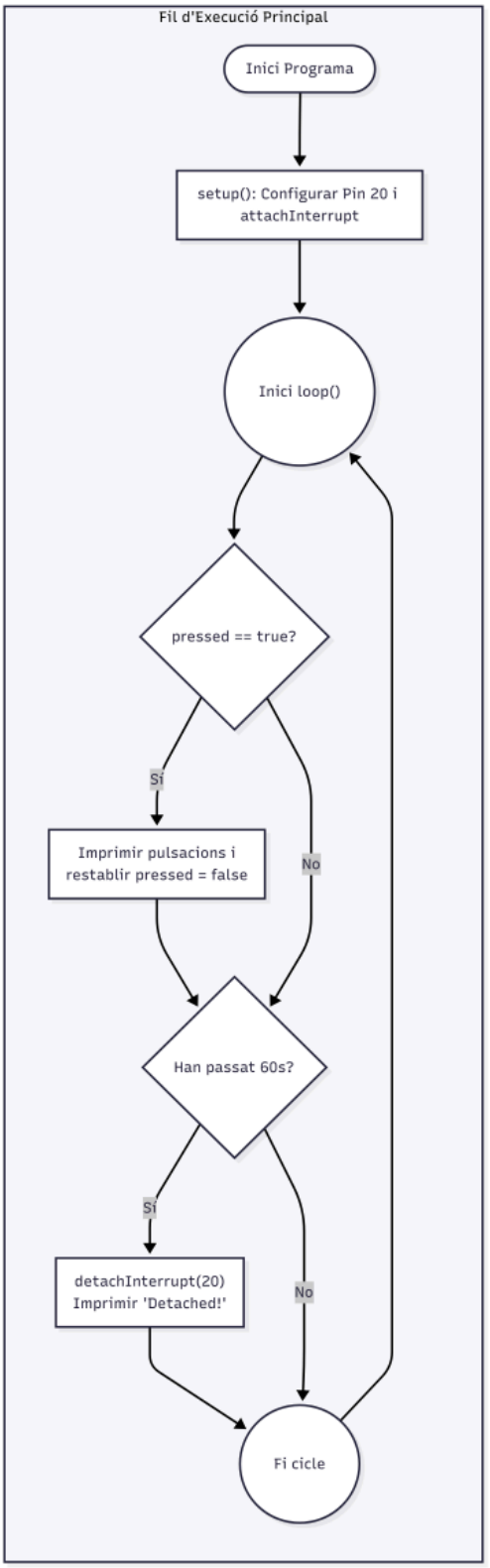
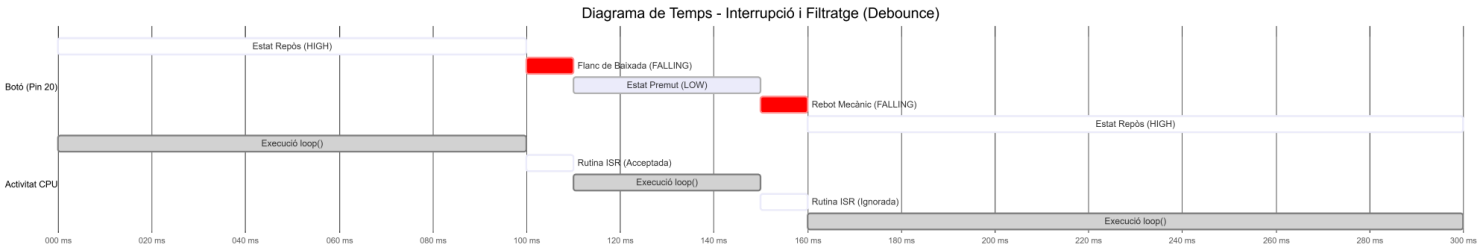


Diagrama de temps



Desenvolupament i Implementació: Pràctica B - Interrupció per Timer

En aquesta segona part, l'objectiu és comprendre el funcionament de les interrupcions programades per temps o temporitzadors (Timers). A diferència del botó, on no sabem quan es produirà l'esdeveniment, un temporitzador genera una interrupció de forma autònoma i exacta cada vegada que el seu comptador intern assoleix un valor predeterminat.

1. Configuració de l'Entorn (platformio_2.ini)

L'arxiu de configuració és pràcticament idèntic al de la pràctica anterior. Manté la definició de la placa i del framework d'Arduino:

- **monitor_speed = 115200**: Aquesta línia continua sent indispensable. Fixa la velocitat del port sèrie a 115200 bauds perquè l'ESP32 i l'ordinador es comuniquin correctament. Si aquest valor no quadra amb el del codi, la sortida pel terminal es veuria com a caràcters incomprensibles.

2. Codi Principal: Interrupció per Timer (main.cpp)

Aquest codi implementa un comptador de segons exacte basat en el maquinari del processador, el qual es pot aturar prement el botó físic.

Variables i Estructures de control:

- **volatile int interruptCounter;**: El modificador volatile és crucial aquí. Li diu al compilador que aquesta variable pot canviar en qualsevol moment (a causa de la interrupció), evitant que l'optimitzi o l'amagui a la memòria cau.
- **hw_timer_t * timer = NULL;**: Es crea un punter per emmagatzemar la configuració del temporitzador de maquinari.
- **portMUX_TYPE timerMux = ...;**: Crea un mecanisme de bloqueig (Mutex o *semàfor*). Serveix per evitar que el bucle principal i la interrupció intentin llegir i escriure la mateixa variable exactament al mateix temps, fet que corrompria les dades.

Funció d'Interrupció del Temporitzador (ISR):

- **void IRAM_ATTR onTimer() { ... }**: Igual que amb el botó, s'allotja a la memòria RAM per a una execució ultraràpida.
- **portENTER_CRITICAL_ISR(&timerMux); i portEXIT_CRITICAL_ISR(&timerMux);**: Aquestes línies "bloquegen" l'accés a les variables mentre la interrupció incrementa el comptador (interruptCounter++), garantint l'exclusivitat de l'operació.

Funció setup() (Configuració del Timer): El processador ESP32-S3 té un rellotge base de 80 MHz.

- **timer = timerBegin(0, 80, true);**: Inicia el Timer número 0. El valor 80 és el *prescaler* (divisor). Al dividir 80 MHz entre 80, el temporitzador avança a 1 MHz, és a dir, fa 1 "tic" cada microsegon.
- **timerAttachInterrupt(timer, &onTimer, true);**: Vincula la funció onTimer perquè s'executi quan l'alarma salti.
- **timerAlarmWrite(timer, 1000000, true);**: Configura el límit de l'alarma. Com que cada "tic" és un microsegon, s'introdueix el valor 1.000.000 perquè l'alarma salti exactament cada 1 segon. El true final indica que es reiniciï automàticament.
- **timerAlarmEnable(timer);**: Engega el temporitzador.

Funció loop(): El bucle principal s'encarrega d'imprimir els valors i de gestionar la parada.

- **Gestió del conteo (if (interruptCounter > 0))**: Si la interrupció ha saltat, el bucle principal entra en una zona crítica (portENTER_CRITICAL), descompta la bandera d'interrupció i incrementa el comptador total de segons (totalInterruptCounter++).
- **Gestió de la parada (timerAlarmDisable(timer))**: Si es detecta que s'ha premut el botó (reaprofitant el codi de la Pràctica A), el programa crida aquesta funció que atura físicament l'alarma del temporitzador, posant fi al conteo.

Sortida pel Terminal de Control

En executar el programa, el comportament visualitzat al monitor sèrie és el d'un cronòmetre precís. Es mostra la següent seqüència:

1. Només engegar, apareix el missatge **--- Sistema Iniciado: Contando ---**.
2. Cada segon exacte, el terminal va mostrant: **Tiempo: 1 Tiempo: 2 Tiempo: 3**... i així successivament.
3. En el moment exacte en què l'usuari prem el botó, el temporitzador s'atura gràcies a l'enllaç entre les dues interrupcions, i es mostra **pel terminal de sortida**: **!!! BOTÓN PULSADO !!! El temporizador ha sido detenido. Conteo final: [X] segundos.** (mostrant els segons totals transcorreguts).
4. Si es torna a premer el botó un cop aturat, el programa simplement avisa amb **El temporizador ya estaba detenido.**.

Anàlisi del Temps Lliure del Processador (Interrupció per Timer)

L'anàlisi de l'ocupació de la CPU en aquest segon programa demostra per què els temporitzadors de maquinari són la forma més professional i eficient de gestionar tasques periòdiques.

El problema de mesurar el temps tradicionalment: Si volguéssim fer un cronòmetre senzill o executar una tasca exactament cada segon sense utilitzar interrupcions, normalment recorreríem a la instrucció delay(1000). Com ja vam veure a la Pràctica 1, això bloqueja completament el processador: durant aquell segon d'espera, la CPU és incapaç de llegir sensors, atendre botons o processar dades. A més, l'ús de delay() introdueix errors de precisió, ja que no té en compte el temps que triguen a executar-se les altres instruccions del codi.

L'eficiència del Temporitzador (Hardware Timer): En aquest programa (main.cpp), hem delegat la tasca de "comptar el temps" a un component físic intern del microcontrolador anomenat temporitzador, el qual funciona de forma totalment independent al processador principal. Aquest temporitzador compta els cicles de rellotge en segon pla basant-se en un comptador de 64 bits.

Gràcies a això, l'ocupació del processador es redueix al mínim absolut:

- **Treball en segon pla:** Mentre el temporitzador avança inexorablement microsegon a microsegon fins a arribar al milió (1 segon), la CPU no hi intervé per a res. Està executant el bucle loop() a màxima velocitat avaluant els condicionals sense aturar-se.
- **Temps d'execució de la interrupció (ISR): < 0,1%.** Un cop per segon, el temporitzador assoleix el valor límit i dispara l'alarma (interrupció). La CPU pausa el bucle principal només durant una fracció minúscula de mil·lisegon per executar la funció onTimer(), incrementar la variable interruptCounter a la memòria RAM, i tornar a la seva tasca.
- **Temps de processament al loop:** La CPU dedica uns pocs mil·lisegons a enviar la dada actualitzada pel terminal sèrie només quan detecta que la variable ha canviat.
- **Temps lliure: > 99,9%.**

Diagrames de flux i temps (B)

Diagrama de flux

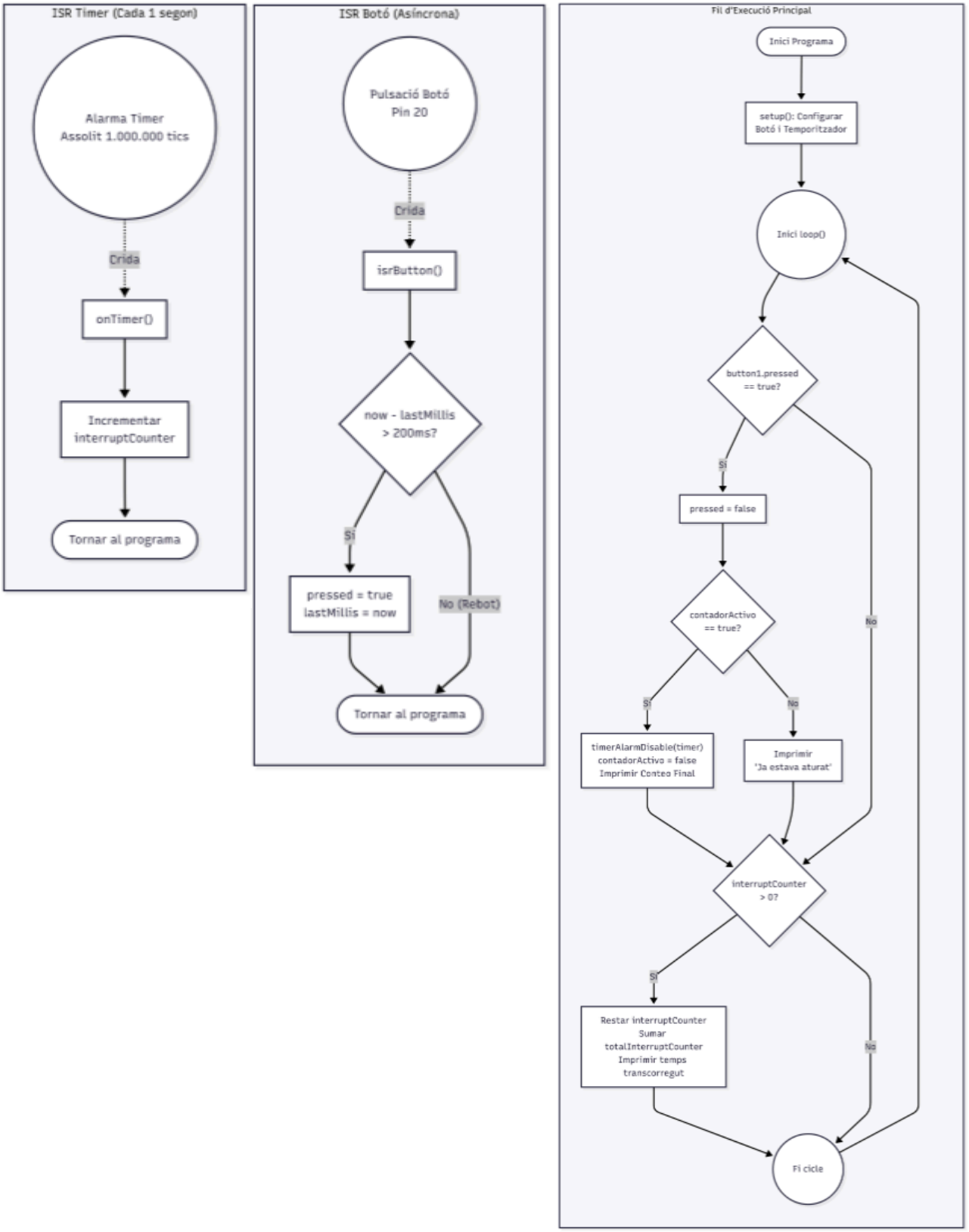
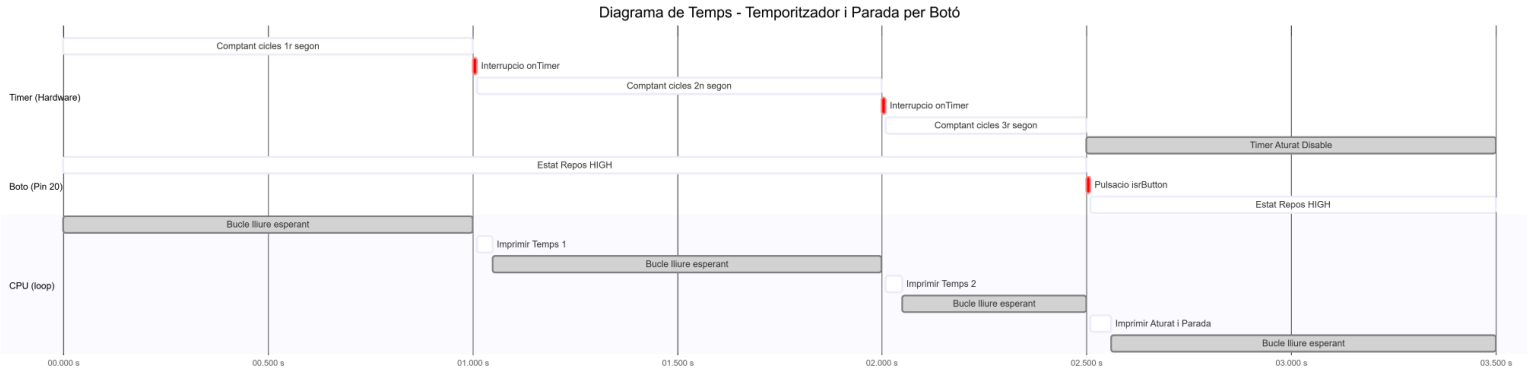


Diagrama de temps



Desenvolupament i Implementació: Pràctica EXTRA (Codi Generat per IA)

Aquesta pràctica extra s'ha desenvolupat a partir d'un codi generat per ChatGPT, complint amb l'objectiu de controlar la freqüència de parpelleig d'un LED mitjançant dos botons, gestionant-ho tot exclusivament a través de les interrupcions d'un temporitzador (Timer) i incorporant un filtre per programari per als rebots.

1. Configuració de l'Entorn (platformio_3.ini)

El fitxer de configuració manté els paràmetres estàndard de la placa. Com en els casos anteriors, l'única línia crítica per al desenvolupament d'aquest codi és:

- **monitor_speed = 115200**: Defineix la velocitat de transmissió del port sèrie a 115200 bauds, necessària per poder llegir la sortida del terminal correctament a l'ordinador.

2. Codi Principal: Temporitzador i Lògica de Botons (main.cpp)

L'arquitectura d'aquest programa és molt robusta i funciona gairebé com un petit sistema operatiu de temps real. Configura un únic temporitzador perquè salti cada 1 mil·lisegon i, a cada "tic", comprova l'estat del LED i dels botons.

Estructures de Dades i Variables Compartides:

- **struct ButtonState { ... };**: ChatGPT ha creat una estructura per emmagatzemar tot l'estat d'un botó. Inclou el pin, un comptador d'estabilitat (stableCount), l'última lectura (lastReading), l'estat actual (state) i una bandera per avisar al bucle principal (actionPending).
- **volatile int currentPeriodMs = 500;**: Aquesta variable marca la velocitat actual del parpelleig en mil·lisegons. S'inicialitza a 500 ms (mig segon).

- **portMUX_TYPE timerMux = ...;**: Tal com vam veure a la Pràctica B, aquesta línia crea un forrellat (mutex) per protegir les variables compartides entre el bucle principal i la interrupció.

Configuració del Temporitzador (setup()):

- **timerAlarmWrite(timer, TIMER_INTERVAL_US, true);**: Atès que TIMER_INTERVAL_US està definit a 1000, el temporitzador es configura perquè l'alarma salti exactament cada 1000 microsegons (1 mil·lisegon).

Rutina de Servei d'Interrupció (ISR - onTimer i handleButtonLogic): Aquesta és la part més complexa i eficient del codi. A diferència de la Pràctica A (on el botó generava la interrupció), aquí és el temporitzador qui, un cop cada mil·lisegon, "pregunta" als botons com estan.

- **Lògica del LED**: Dins de onTimer(), un comptador (blinkCounter) s'incrementa cada mil·lisegon. Quan assoleix el valor de currentPeriodMs, es reinicia i inverteix l'estat del pin del LED (encenent-lo o apagant-lo).
- **Lògica Antirebots (Debounce)**: La funció handleButtonLogic() llegeix el voltatge del pin i el compara amb la lectura del mil·lisegon anterior. Si el valor canvia (posible rebot), el comptador d'estabilitat es reinicia a zero (btn.stableCount = 0). Si el senyal es manté estable durant 50 mil·lisegons (DEBOUNCE_TIME_MS), es considera una pulsació ferma i real.
- Quan es confirma la pulsació i es detecta un nivell BAIX (LOW), s'activa la bandera btn.actionPending = true;.

Funció Principal (loop()): El bucle principal ja no s'encarrega d'esperar o de comptar el temps, sinó només de reaccionar si l'interrupció li demana.

- Si detecta que s'ha de pujar la freqüència (btnUp.actionPending), entra a una secció crítica (portENTER_CRITICAL) i redueix la variable currentPeriodMs restant-li 100 ms (STEP_MS), fent que el LED parpellegi més ràpid.
- Si detecta el botó de baixar freqüència, li suma 100 ms a la variable, fent el parpelleig més lent.
- Finalment, hi ha un mecanisme de seguretat per evitar que el període baixi de 50 ms o superi els 2000 ms.

Sortida pel Terminal de Control

L'execució del programa ofereix una resposta immediata i visual pel terminal sèrie per confirmar l'èxit de les operacions:

1. En iniciar el microcontrolador, es mostra el missatge d'arrancada:
   Sistema iniciado. Timer corriendo a 1ms.

2. Quan es prem de forma estable el polsador associat a pujar la freqüència (Pin 40), el terminal mostra:
   Frecuencia Aumentada (Periodo menor). Periodo actual: 400 ms

3. Si es prem l'altre polsador (Pin 20), el sistema reacciona i imprimeix: **Frecuencia Disminuida (Periodo mayor). Periodo actual: 500 ms**

Aquest disseny garanteix que la sortida pel terminal només es produeixi una única vegada per pulsació gràcies a l'efectivitat del filtre antirebots programat a la interrupció.

Anàlisi del Temps Lliure del Processador (Pràctica EXTRA)

El model d'execució d'aquest codi generat per IA presenta un paradigma molt comú en sistemes integrats professionals: l'ús d'un **temporitzador de mostreig continu** (conegut com a *Tick Timer* o *System Tick*).

L'impacte de les interrupcions d'alta freqüència: En aquest programa, l'alarma del temporitzador no salta cada segon com a la Pràctica B, sinó que està configurada per saltar **cada 1 mil·lisegon** (1000 vegades per segon). Això podria fer pensar que el processador està sobrecarregat atès que ha d'interrompre el seu flux constantment.

No obstant això, si analitzem la funció de la interrupció (onTimer i handleButtonLogic), veiem que només realitza operacions matemàtiques bàsiques (sumar un a un comptador) i avaluacions booleanes (llegir un pin i comparar true/false). Aquestes instruccions es processen en un marge de pocs microsegons o nanosegons.

- **Temps d'execució de la ISR:** Tot i que salta 1000 vegades per segon, la suma total de temps que la CPU dedica a atendre aquestes interrupcions al llarg d'un segon és pràcticament insignificant (estimat en un 1% o 2% de l'ús total).

El comportament del fil principal: El bucle principal (loop()) es troba totalment alliberat de la responsabilitat de vigilar els rebots o comptar el temps per fer parpellejar el LED. La seva única feina és comprovar si les banderes btnUp.actionPending o btnDown.actionPending han estat activades per la interrupció. Si no s'ha premut cap botó, el bucle ignora les condicions i arriba a la instrucció **delay(10);**.

- **Temps lliure (espera): > 95%.** L'ús d'aquest petit retard de 10 mil·lisegons indica que la CPU s'atura intencionadament durant la major part del cicle principal.

Conclusions

Aquesta segona pràctica ha suposat un salt qualitatiu molt important en la manera d'entendre i dissenyar codi per a microcontroladors. S'han assolit amb èxit tots els objectius plantejats, passant d'una programació seqüencial i bloquejant a una arquitectura basada en esdeveniments. Les conclusions principals que se n'extreuen són les següents:

- **Superació del model de *Polling*:** S'ha demostrat pràcticament que revisar contínuament l'estat d'un pin (*polling*) és ineficient i pot provocar la pèrdua de dades si el processador està ocupat. La implementació d'interrupcions per maquinari

(GPIO) ha permès que el sistema reaccioni de forma fulgurant als estímuls físics (com la pulsació d'un botó), alliberant el processador d'aquesta tasca de vigilància.

- **Precisió absoluta amb Temporitzadors (Timers):** Mitjançant l'ús del maquinari intern de l'ESP32, s'ha aconseguit comptar el temps de forma exacta i paral·lela a l'execució del programa. Això elimina la dependència de funcions imprecises i bloquejants com delay(), obrint la porta a desenvolupar aplicacions que requereixin una temporització crítica.
- **Gestió de problemes físics i concurrència:** Un dels grans aprenentatges pràctics ha estat el tractament dels senyals físics. S'ha hagut d'implementar un filtre antirebots (*debounce*) per programari per ignorar les imperfeccions mecàniques dels polsadors. A més, en treballar amb ISR (Rutines de Servei d'Interrupció) i el bucle principal simultàniament, s'ha comprès la necessitat vital d'utilitzar seccions crítiques (Mutex) per protegir les variables compartides i evitar la corrupció de dades.
- **Optimització extrema del temps de procés:** L'anàlisi de temps realitzat al llarg dels diferents exercicis demostra que, gràcies a les interrupcions, el processador gaudeix de més d'un 99% de temps lliure real i aprofitable. Fins i tot en el disseny avançat generat per intel·ligència artificial, on el temporitzador salta 1000 vegades per segon, la càrrega de la CPU és mínima gràcies al fet que les instruccions dins de la ISR són extremadament ràpides i precises.

En definitiva, el domini de les interrupcions és una eina fonamental per a qualsevol enginyer que vulgui dissenyar sistemes encastats (*embedded systems*) professionals. Aquesta pràctica ens dota de les eines necessàries per programar sistemes en temps real, capaços d'atendre múltiples sensors, comunicacions i actuadors de forma concurrent i sense pèrdues de rendiment.